

Perancangan dan Implementasi Sistem Manajemen Kost Berbasis Command Line Interface (CLI) Menggunakan Python

Disusun untuk memenuhi tugas akhir Project-Based Learning (PBL)

Mata Kuliah Algoritma & Struktur Data (TPL2106)

Disusun oleh:

Nama	NIM
Muhammad Rafly	J0403251002
Lumaris Satya Dwinanto	J0403251143
Fahrizal Azis	J0403251151



Program Studi Teknologi Rekayasa Perangkat Lunak

Sekolah Vokasi IPB University

2026

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan laporan proyek yang berjudul "Perancangan dan Implementasi Sistem Manajemen Kost-Kostan Berbasis Command Line Interface (CLI) Menggunakan Python" dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu bentuk dokumentasi dari hasil pengembangan aplikasi manajemen kost-kostan yang dibuat menggunakan bahasa pemrograman Python. Aplikasi ini dirancang untuk membantu proses pengelolaan data kost, data penghuni, serta berbagai informasi terkait secara lebih terstruktur dan efisien melalui antarmuka Command Line Interface (CLI).

Penulis menyadari bahwa laporan ini masih memiliki berbagai kekurangan, baik dari segi isi maupun penyajiannya. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan dan pengembangan di masa yang akan datang.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat serta menambah wawasan bagi pembaca mengenai pengembangan aplikasi berbasis Python, khususnya dalam bidang pengelolaan kost-kostan.

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	2
1.1 Latar Belakang	2
1.2 Tujuan Proyek	2
1.3 Ruang Lingkup.....	3
BAB II ANALISIS DAN PERANCANGAN	4
2.1 Deskripsi Sistem	4
2.2 Struktur Data yang Digunakan.....	4
2.2.1 Fungsi dari Struktur Data yang Digunakan	4
2.2.3 Alasan Pemilihan Struktur Data	5
2.3 Algoritma yang Digunakan	5
2.4 Diagram Alir Program (Flowchart)	6
2.5 Struktur Program.....	8
BAB III. IMPLEMENTASI PROGRAM	10
3.1 Tampilan Menu Utama	10
3.2 Fitur Create	10
3.3 Fitur Read	11
3.4 Fitur Update.....	12
3.5 Fitur Delete	13
3.6 Fitur Searching	13
3.7 Fitur Sorting	14
3.8 Penyimpanan Data (File Handling)	15
3.9 Validasi Input dan Error Handling	15
BAB IV. PENGUJIAN DAN ANALISIS.....	17
4.1 Skenario Pengujian	17
4.2 Kelebihan Sistem	18
4.3 Keterbatasan Sistem.....	18
BAB V. KENDALA DAN SOLUSI	19
5.1 Tabel Kendala dan Solusi	19
5.2 Pengalaman Menantang Selama Pengerjaan Proyek.....	20
BAB VI PEMBAGIAN TUGAS ANGGOTA	21
BAB VII. KESIMPULAN DAN SARAN.....	22
7.1 Kesimpulan.....	22
7.2 Saran Pengembangan.....	22
DAFTAR PUSTAKA	23
LAMPIRAN	24

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pengelolaan data kost-kostan sering kali masih dilakukan secara manual menggunakan buku catatan atau spreadsheet sederhana. Cara tersebut berisiko menyebabkan kesalahan pencatatan, kehilangan data, serta menyulitkan proses pencarian, pengubahan, dan pengelolaan informasi kamar maupun penghuni kost. Menurut Kadir (2014), sistem informasi berperan dalam mengolah data menjadi informasi yang berguna untuk mendukung pengambilan keputusan dan meningkatkan efisiensi pengelolaan data.

Proyek ini dibuat untuk membantu pemilik kost dalam mengelola data kamar dan penghuni secara lebih terstruktur dan efisien. Sistem mampu menyimpan data secara permanen ke dalam file CSV serta menyediakan fitur CRUD (Create, Read, Update, Delete), pencarian data, pengurutan data, dan undo perubahan. Dalam implementasinya, sistem menggunakan struktur data Single Linked List untuk mengelola data kamar dan Stack untuk mendukung fitur undo sehingga pengelolaan data menjadi lebih efektif dan sesuai dengan konsep struktur data yang dipelajari.

1.2 Tujuan Proyek

1. Membuat aplikasi manajemen kost-kostan berbasis terminal (CLI) menggunakan bahasa pemrograman Python.
2. Mengimplementasikan struktur data Single Linked List untuk pengelolaan data kamar dan Stack untuk fitur undo.
3. Mengimplementasikan algoritma Sequential Search, Traversal Linked List, dan Bubble Sort dalam pengolahan data.
4. Menerapkan konsep file handling menggunakan file CSV agar data dapat disimpan secara permanen.
5. Membantu pemilik kost dalam mengelola data kamar dan penghuni secara lebih terstruktur, cepat, dan efisien.

1.3 Ruang Lingkup

- Platform CLI (Console)
- Bahasa pemrograman Python.
- Penyimpanan data menggunakan file CSV.
- Fitur CRUD (Create, Read, Update, Delete) data kamar.
- Fitur Searching
- Fitur Sorting
- Fitur Undo

BAB II

ANALISIS DAN PERANCANGAN

2.1 Deskripsi Sistem

Aplikasi Manajemen Kosst-kostan ini merupakan aplikasi berbasis Command Line Interface (CLI) yang dibuat menggunakan bahasa pemrograman Python untuk membantu pemilik kost dalam mengelola data kamar dan penghuni. Sistem ini menyediakan fitur untuk menambahkan, menampilkan, mengubah, menghapus, mencari, dan mengurutkan data kamar sehingga proses pengelolaan data dapat dilakukan secara lebih terstruktur dan efisien.

Data yang dikelola meliputi nomor kamar, lantai, harga kamar, status kamar, nama penghuni, tanggal mulai sewa, dan tagihan. Sistem juga dilengkapi dengan fitur undo yang memungkinkan pengguna membatalkan perubahan terakhir yang dilakukan pada data.

Untuk menjaga keberlangsungan data, sistem menggunakan file CSV sebagai media penyimpanan permanen. Dengan demikian, data yang telah disimpan tetap tersedia dan dapat digunakan kembali ketika program dijalankan pada waktu berikutnya. Sistem ini juga menyediakan fitur pencarian berdasarkan nomor kamar atau nama penghuni, pengurutan data kamar berdasarkan nomor kamar menggunakan algoritma Bubble Sort, serta pencatatan riwayat aktivitas pengguna selama program berjalan. Seluruh data disimpan ke dalam file CSV sehingga data tetap tersedia dan dapat digunakan kembali ketika program dijalankan pada waktu berikutnya.

2.2 Struktur Data yang Digunakan

2.2.1 Fungsi dari Struktur Data yang Digunakan

Struktur Data	Fungsi
Single Linked List	Digunakan sebagai struktur data utama untuk menyimpan dan mengelola data kamar kost. Setiap data kamar disimpan dalam sebuah node yang saling terhubung sehingga mendukung proses penambahan, pencarian, pembaruan, dan penghapusan data.
Stack	Digunakan untuk menyimpan riwayat kondisi data sebelum terjadi perubahan. Struktur data ini mendukung fitur Undo sehingga pengguna dapat mengembalikan data ke kondisi sebelumnya.

List	Digunakan sebagai media penyimpanan sementara hasil pembacaan data dari file CSV serta sebagai jembatan antara file CSV dan Single Linked List.
------	---

2.2.3 Alasan Pemilihan Struktur Data

- **Single Linked List**

Single Linked List dipilih karena mampu menyimpan data secara dinamis tanpa harus menentukan ukuran penyimpanan di awal. Selain itu, struktur data ini memudahkan proses penambahan, pencarian, pembaruan, dan penghapusan data kamar melalui mekanisme penelusuran node yang saling terhubung.

- **Stack**

Stack dipilih karena menerapkan prinsip Last In First Out (LIFO), yaitu data yang terakhir disimpan akan menjadi data pertama yang diambil kembali. Prinsip ini sangat sesuai untuk implementasi fitur Undo karena perubahan terakhir yang dilakukan pengguna dapat dibatalkan terlebih dahulu.

- **List**

List dipilih karena mudah digunakan untuk menampung data yang dibaca dari file CSV dan mendukung proses penyimpanan kembali ke file CSV. Selain itu, List memudahkan sinkronisasi data antara file penyimpanan dan struktur data Single Linked List yang digunakan dalam sistem.

2.3 Algoritma yang Digunakan

- **Searching (Sequential Search)**

Alasan Penggunaan:

Sequential Search digunakan untuk mencari data kamar berdasarkan nomor kamar atau nama penghuni. Algoritma ini bekerja dengan memeriksa data satu per satu secara berurutan mulai dari node pertama hingga data yang dicari ditemukan atau seluruh data telah diperiksa. Sequential Search dipilih karena data disimpan menggunakan struktur data Single Linked List yang tidak mendukung akses langsung ke elemen tertentu seperti array. Oleh karena itu, pencarian harus dilakukan dengan menelusuri setiap node secara berurutan.

- **Sorting (Bubble Sort)**

Alasan Penggunaan:

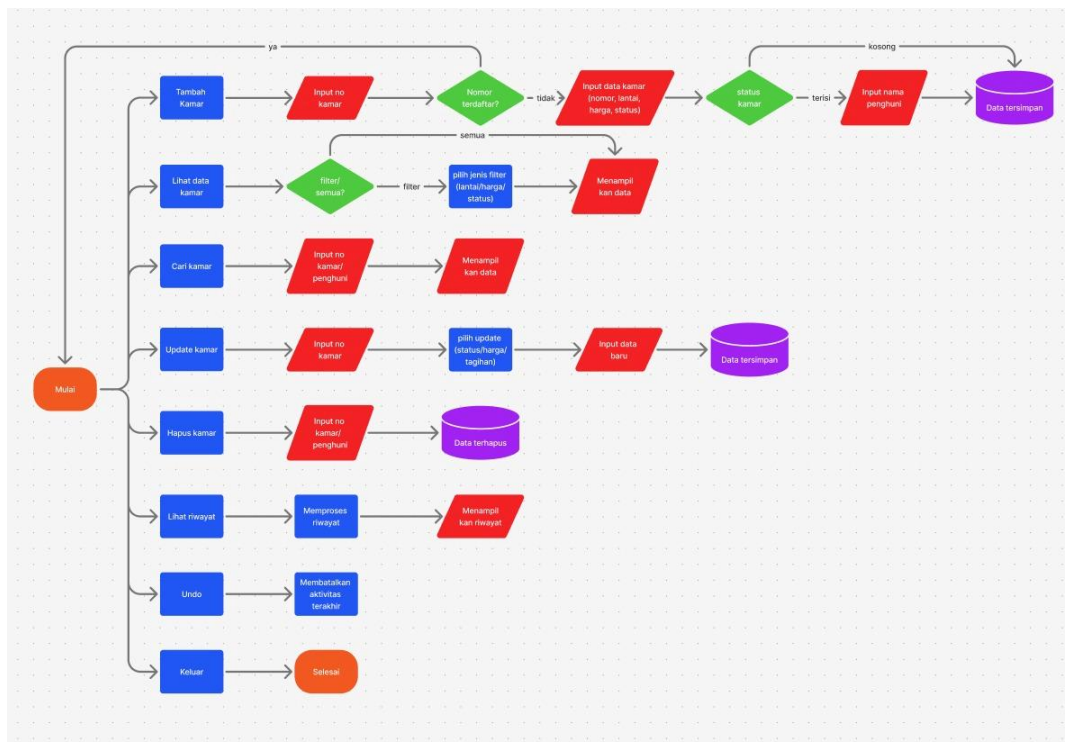
Bubble Sort digunakan untuk mengurutkan data kamar berdasarkan nomor kamar dari nilai terkecil ke terbesar. Algoritma ini bekerja dengan membandingkan dua data yang berdekatan dan menukarnya apabila urutannya tidak sesuai. Bubble Sort dipilih karena mudah diimplementasikan pada struktur data Single Linked List serta sesuai untuk jumlah data yang relatif kecil sehingga proses pengurutan tetap dapat dilakukan dengan baik.

- **Traversal Linked List**

Alasan Penggunaan:

Traversal Linked List digunakan untuk menelusuri seluruh node yang terdapat dalam Single Linked List. Algoritma ini diterapkan pada proses menampilkan data kamar, mencari data kamar, memperbarui data kamar, dan menghapus data kamar. Traversal diperlukan karena setiap node pada Linked List hanya mengetahui node berikutnya sehingga data harus ditelusuri secara berurutan mulai dari head hingga node terakhir.

2.4 Diagram Alir Program (Flowchart)



Gambar 1 Flowchart Aplikasi Manajemen kost-kostan

Flowchart Sistem Manajemen Kost-Kostan dimulai dari beberapa pilihan menu utama, yaitu Tambah Kamar, Lihat Data Kamar, Cari Kamar, Update Kamar, Hapus Kamar, Lihat Riwayat, Undo, dan Keluar.

1. Tambah Kamar

Pada menu ini, pengguna memasukkan nomor kamar terlebih dahulu. Sistem akan melakukan validasi untuk memastikan nomor kamar belum terdaftar. Jika nomor kamar sudah ada, pengguna diminta memasukkan nomor kamar yang lain. Jika nomor kamar belum terdaftar, pengguna dapat memasukkan data kamar seperti nomor kamar, lantai, harga, dan status kamar. Apabila status kamar adalah Terisi, pengguna harus mengisi data penghuni. Setelah seluruh data lengkap, data akan disimpan ke dalam sistem dan file CSV.

2. Lihat Data Kamar

Pengguna dapat memilih untuk menampilkan seluruh data kamar atau menggunakan filter berdasarkan lantai, harga, maupun status kamar. Setelah pilihan filter ditentukan, sistem akan menampilkan data kamar yang sesuai dengan kriteria yang dipilih.

3. Cari Kamar

Pengguna memasukkan nomor kamar atau nama penghuni yang ingin dicari. Sistem kemudian melakukan proses pencarian menggunakan algoritma Sequential Search dan menampilkan data yang ditemukan.

4. Update Kamar

Pengguna memasukkan nomor kamar yang akan diperbarui. Setelah data ditemukan, pengguna dapat memilih data yang ingin diubah, yaitu status dan penghuni, tagihan, atau harga kamar. Setelah perubahan dilakukan, data baru akan disimpan ke dalam sistem.

5. Hapus Kamar

Pengguna memasukkan nomor kamar yang akan dihapus. Sistem akan mencari data tersebut dan menghapusnya dari struktur data serta file penyimpanan.

6. Lihat Riwayat

Sistem menampilkan daftar aktivitas yang telah dilakukan pengguna selama program berjalan, seperti penambahan, perubahan, penghapusan, maupun pengurutan data.

7. Undo

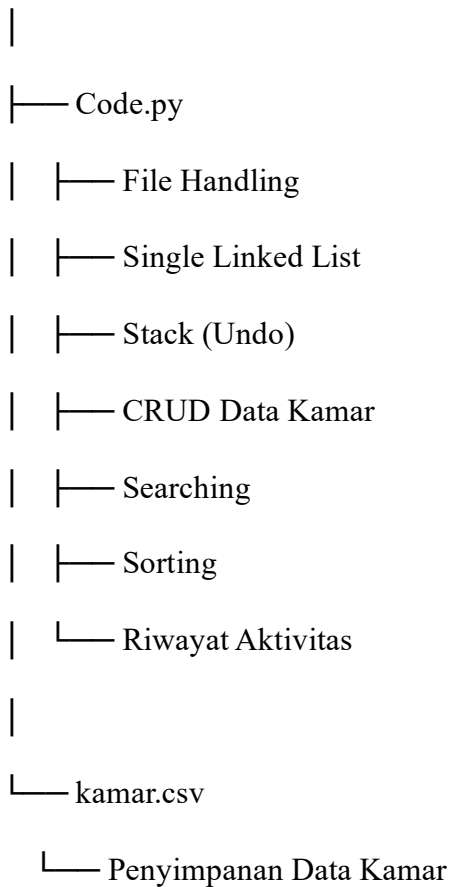
Sistem mengambil data terakhir yang tersimpan pada Stack dan mengembalikannya ke kondisi sebelum perubahan dilakukan. Fitur ini digunakan untuk membatalkan perubahan terakhir yang dilakukan pengguna.

8. Keluar

Jika pengguna memilih menu keluar, program akan menghentikan seluruh proses dan masuk ke kondisi Selesai.

2.5 Struktur Program

Sistem Manajemen Kost-Kostan



Penjelasan:

Code.py → File utama yang berisi seluruh kode program Sistem Manajemen Kost-Kostan. File ini mencakup implementasi fitur CRUD (Create, Read, Update, Delete), pencarian data kamar, pengurutan data, fitur undo, riwayat aktivitas, pengelolaan file CSV, serta implementasi struktur data Single Linked List dan Stack. Program dijalankan melalui file ini dan seluruh menu aplikasi tersedia di dalamnya.

Kamar.csv → File penyimpanan data permanen yang digunakan oleh sistem. File ini menyimpan seluruh data kamar kost yang meliputi nomor kamar, lantai, harga, status kamar, nama penghuni, tanggal mulai sewa, dan tagihan.

BAB III

IMPLEMENTASI PROGRAM

3.1 Tampilan Menu Utama

Aplikasi dijalankan melalui file Code.py. Saat pertama kali program dieksekusi, fungsi `load_data()` akan otomatis membaca file `kamar.csv`, kemudian memanggil `sll.rebuild(data)` untuk menyusun seluruh baris data menjadi rantai *Node* di dalam memori. Setelah itu, sistem akan menampilkan 9 pilihan menu interaktif berbasis *Command Line Interface* (CLI) sebagai berikut:

```
===== MENU APLIKASI =====
1. Tambah Kamar
2. Lihat Semua Kamar
3. Cari Kamar (No/Penghuni)
4. Update Kamar (Nomor)
5. Hapus Kamar (Nomor)
6. Lihat Riwayat
7. Undo
8. Urutkan Kamar (Bubble Sort)
0. Keluar

Pilih menu: █
```

Gambar 2: Menu Aplikasi

3.2 Fitur Create

Fitur ini diimplementasikan melalui fungsi `tambah_kamar(sll)`. Ketika pengguna ingin menambahkan aset kamar baru, sistem akan meminta input nomor kamar terlebih dahulu. Sebelum meminta input data lainnya, sistem akan menjalankan fungsi validasi nomor unik dengan melakukan *traversal* pada Linked List. Jika lolos validasi, pengguna memasukkan data lantai, harga, status, dan nama penghuni. Objek baru dibungkus ke dalam `Node(kamar)` dan disambungkan ke ujung akhir rantai melalui metode `sll.tambah_kamar(kamar)`.

```

Pilih menu: 1
Nomor kamar: 01
Lantai: 1
Harga: 1750000
Status (Terisi/Kosong): Terisi
Penghuni: Bunatedy
Tagihan pembayaran (Rp): 990000
Kamar berhasil ditambahkan! (Tanggal mulai: 2026-06-26 )

```

Gambar 3 Fitur Create

3.3 Fitur Read

Fitur ini ditangani oleh fungsi `lihat_kamar()`. Pengguna diberikan dua opsi tampilan, yaitu menampilkan keseluruhan data kamar yang aktif atau melakukan penyaringan (*filtering*). Proses penyaringan dapat dikelompokkan berdasarkan parameter lantai, harga, atau status hunian (Terisi/Kosong). Tampilan data disajikan dalam bentuk tabel grid yang rapi di terminal memanfaatkan pustaka `tabulate`.

```
Lihat semua kamar atau berdasarkan filter? (semua/filter): semua
```

```
===== DATA KAMAR =====
```

Nomor	Lantai	Harga	Status	Penghuni	Tanggal Mulai	Tagihan (Rp)
1	1	10	Terisi	Jokowi	2026-06-21	20
01	1	1750000	Terisi	Bunatedy	2026-06-26	990000
02	1	20000000	Kosong			
03	2	250000000	Terisi	Pria Solo	2026-06-26	1000000

Gambar 4 Ftiur Read (1)

```

Pilih menu: 2
Lihat semua kamar atau berdasarkan filter? (semua/filter): filter

Filter berdasarkan:
1. Lantai
2. Harga
3. Status

Pilih filter (1/2/3): 3
Masukkan status (Terisi/Kosong): Terisi

===== DATA KAMAR =====

```

Nomor	Lantai	Harga	Status	Penghuni	Tanggal Mulai	Tagihan (Rp)
1	1	10	Terisi	Jokowi	2026-06-21	20
01	1	1750000	Terisi	Bunatedy	2026-06-26	990000
03	2	250000000	Terisi	Pria Solo	2026-06-26	1000000

Gambar 5 Fitur Read (2)

3.4 Fitur Update

Fitur ini diimplementasikan dalam fungsi `update_kamar(sll)`. Alur kerjanya dimulai dengan mencari nomor kamar target menggunakan pointer bantuan `temp = sll.head`. Pointer akan bergeser maju melalui `temp = temp.next` hingga menemukan nomor kamar yang cocok. Setelah ditemukan, sistem melakukan *deep copy* data lama ke dalam `history_stack` untuk kebutuhan *undo*. Kemudian, pengguna dapat memasukkan nama penghuni baru, dan kolom status kamar secara otomatis akan diperbarui menjadi "Terisi".

```

Pilih menu: 4
Masukkan Nomor kamar yang mau diupdate: 02

Pilih yang ingin diupdate:
1. Status & Penghuni
2. Tagihan
3. Harga

Pilihan (1/2/3): 1
Status baru (Terisi/Kosong): Terisi
Nama penghuni baru: Herlambang
Data berhasil diupdate langsung di Linked List!

```

Gambar 6 Fitur Update

3.5 Fitur Delete

Proses penghapusan diatur oleh fungsi `hapus_kamar(sll)` yang bekerja sama dengan metode internal `sll.hapus_node_by_nomor(nomor_cari)`. Fitur ini menerapkan manipulasi pointer Linked List yang sesungguhnya dengan menggunakan dua buah pointer bantuan, yaitu `temp` (node saat ini) dan `prev` (node sebelumnya). Jika data yang dihapus berada di tengah atau akhir rantai, sistem akan memutuskan hubungan node target dengan cara mengarahkan pointer `prev.next` langsung melompati node `temp` dan menyambungkannya ke `temp.next`.

```
Pilih menu: 5
Masukkan Nomor kamar yang dihapus: 01
Data berhasil dihapus dari Linked List!
```

Gambar 7 Fitur Delete

3.6 Fitur Searching

Fitur pencarian diakomodasi oleh fungsi `cari_kamar(sll)`. Algoritma yang digunakan adalah **Sequential Search** (Pencarian Berurutan) yang berjalan langsung di atas objek *Single Linked List*. Pointer pembantu `temp` diinisialisasi dari `sll.head` dan menelusuri setiap node satu per satu secara sekuensial sampai mencapai `None`. Sistem mencocokkan kata kunci (*keyword*) yang diinput pengguna dengan data nomor kamar atau nama penghuni secara *case-insensitive* menggunakan fungsi `.lower()`. Setiap data yang cocok akan dikumpulkan ke dalam list temporer untuk ditampilkan secara simultan.

```
Pilih menu: 3
Masukkan nomor kamar atau nama penghuni yang ingin dicari: Pria solo
```

===== Hasil Pencarian =====

Nomor	Lantai	Harga	Status	Penghuni	Tanggal Mulai	Tagihan (Rp)
03	2	250000000	Terisi	Pria Solo	2026-06-26	1000000

Gambar 8 Fitur Searching

3.7 Fitur Sorting

Fitur pengurutan data diatur oleh fungsi `sort_kamar(sll)` yang memicu metode internal `sll.urutkan_kamar()`. Algoritma yang diimplementasikan adalah **Bubble Sort** yang dimodifikasi khusus untuk struktur data Linked List. Logika algoritma ini bekerja dengan menelusuri rantai memori menggunakan pointer temp dari depan ke belakang untuk membandingkan nilai `int(temp.data["nomor"])` dengan node di depannya, yaitu `int(temp.next.data["nomor"])`. Jika data node saat ini lebih besar dari node berikutnya, program akan melakukan pertukaran isi data (*in-place data swapping*) tanpa merusak susunan alamat pointer next. Proses *looping* akan terus berulang hingga seluruh nomor kamar terurut dari nilai terkecil ke terbesar (*ascending*).

Sebelum Sorting:

===== DATA KAMAR =====						
Nomor	Lantai	Harga	Status	Penghuni	Tanggal Mulai	Tagihan (Rp)
1	1	10	Terisi	Jokowi	2026-06-21	20
02	1	20000000	Terisi	Herlambang	2026-06-26	
03	2	250000000	Terisi	Pria Solo	2026-06-26	1000000
09	2	910000000	Terisi	Windah Basudara	2026-06-26	10
07	1	120000000	Terisi	Mulyadi	2026-06-26	7000000
08	2	350000	Terisi	owok	2026-06-26	120213

Gambar 9 Fitur Sorting (1)

Setelah Sorting:

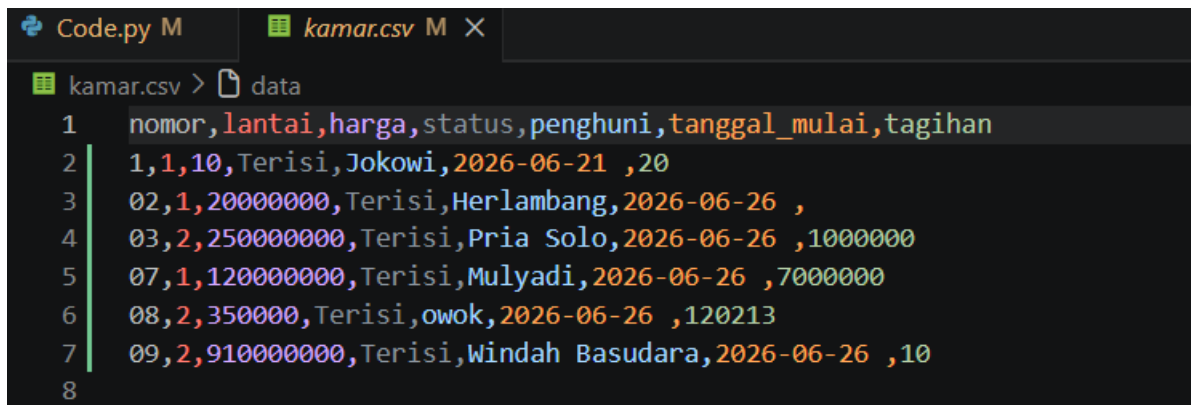
===== DATA KAMAR (TERURUT) =====						
Nomor	Lantai	Harga	Status	Penghuni	Tanggal Mulai	Tagihan (Rp)
1	1	10	Terisi	Jokowi	2026-06-21	20
02	1	20000000	Terisi	Herlambang	2026-06-26	
03	2	250000000	Terisi	Pria Solo	2026-06-26	1000000
07	1	120000000	Terisi	Mulyadi	2026-06-26	7000000
08	2	350000	Terisi	owok	2026-06-26	120213
09	2	910000000	Terisi	Windah Basudara	2026-06-26	10

Gambar 10 Fitur Sorting (2)

3.8 Penyimpanan Data (File Handling)

Sistem ini menggunakan file eksternal berformat CSV (kamar.csv) sebagai media penyimpanan data permanen (*data persistence*).

- **Membaca File:** Menggunakan objek `csv.DictReader(file)` untuk memetakan setiap baris dalam file CSV menjadi bentuk *Dictionary* Python, yang kemudian dipakai untuk mengonstruksi struktur rantai Linked List saat program dimulai.
- **Menyimpan File:** Karena pustaka bawaan `csv.DictWriter` tidak dapat membaca objek Linked List secara langsung, maka dibuat metode `sll.to_list()` di dalam kelas *Single Linked List*. Metode ini bertindak sebagai jembatan untuk mengekstrak seluruh bungkus data dari setiap node menjadi bentuk List standar Python, yang kemudian ditulis ulang ke file CSV melalui metode `.writerows(data)`.



```
Code.py M | kamar.csv M X
kamar.csv > data
1 nomor,lantai,harga,status,penghuni,tanggal_mulai,tagihan
2 1,1,10,Terisi,Jokowi,2026-06-21 ,20
3 02,1,20000000,Terisi,Herlambang,2026-06-26 ,
4 03,2,25000000,Terisi,Pria Solo,2026-06-26 ,1000000
5 07,1,12000000,Terisi,Mulyadi,2026-06-26 ,7000000
6 08,2,350000,Terisi,owok,2026-06-26 ,120213
7 09,2,91000000,Terisi,Windah Basudara,2026-06-26 ,10
8
```

Gambar 11 File CSV

3.9 Validasi Input dan Error Handling

Sistem telah dilengkapi dengan mekanisme validasi dan penanganan kesalahan untuk memastikan aplikasi tetap berjalan stabil dan tidak mengalami *force close (crash)* saat menerima input yang tidak sesuai dari pengguna. Berikut adalah pemetaan kasus kesalahan input beserta penanganannya di dalam sistem:

Kesalahan Input (<i>Error/Exception</i>)	Penanganan (<i>Handling</i>)
Nomor Kamar Duplikat (Pengguna menginput nomor kamar yang sudah terdaftar saat tambah data)	Sistem melakukan <i>traversal</i> pada <i>Single Linked List</i> . Jika nomor ditemukan, program menampilkan pesan error dan langsung membatalkan penambahan data kamar baru.

Pilihan Menu Tidak Valid (Pengguna memasukkan angka di luar 0–8 atau memasukkan karakter huruf pada menu utama)	Sistem mendeteksi pilihan melalui blok pengondisian <code>else</code> . Program akan menampilkan pesan "Pilihan tidak valid!" dan mengarahkan pengguna kembali ke menu utama.
Pilihan Filter Tidak Tersedia (Pengguna memasukkan nomor filter selain 1, 2, atau 3 pada sub-menu lihat kamar)	Blok pengondisian <code>else</code> pada fungsi filter akan langsung menampilkan pesan "Pilihan filter tidak valid!" dan menghentikan proses penampilan data.
Berkas Database (kamar.csv) Belum Tersedia (File penyimpanan eksternal terhapus atau belum dibuat di direktori proyek)	Fungsi <code>load_data()</code> menggunakan penanganan <code>try-except FileNotFoundError</code> . Program akan melewati (<i>pass</i>) pesan error sistem secara aman, dan berkas baru akan otomatis terbuat saat pengguna menyimpan data untuk pertama kalinya.

BAB IV

PENGUJIAN DAN ANALISIS

4.1 Skenario Pengujian

Pengujian dilakukan untuk memastikan seluruh fungsionalitas aplikasi berbasis pemrosesan *Single Linked List* dan *Stack* berjalan sesuai rancangan sistem.

No	Skenario Pengujian	Input yang Dimasukkan	Hasil yang Diharapkan	Hasil Aktual	Keterangan
1	Tambah Data Kamar Baru	Nomor: 103, Lantai: 1, Harga: 1500000, Status: Kosong, Penghuni: -	Data tersimpan di memori SLL dan masuk ke dalam berkas kamar.csv.	Sesuai Harapan	Berhasil
2	Tambah Data Kamar Duplikat	Nomor: 101 (Nomor yang sudah ada di database)	Sistem menampilkan pesan <i>Error</i> dan membatalkan proses input.	Sesuai Harapan	Berhasil
3	Cari Data Kamar (Sequential Search)	Kata kunci: "Tedybear" atau "101"	Menampilkan tabel kamar yang memuat nama atau nomor yang dicari.	Sesuai Harapan	Berhasil
4	Update Data Kamar	Nomor Kamar: 102, Nama Penghuni: Aris	Nama penghuni berubah menjadi "Aris" dan status otomatis menjadi "Terisi".	Sesuai Harapan	Berhasil
5	Hapus Data Kamar	Nomor Kamar: 103	Node kamar terputus dari rantai SLL dan terhapus dari file CSV.	Sesuai Harapan	Berhasil
6	Urutkan Data Kamar	Memilih Menu 8	Susunan baris data di terminal dan file CSV terurut	Sesuai Harapan	Berhasil

	(Bubble Sort)		berdasarkan nomor terkecil.		
7	Pembatalan Aksi terakhir (Undo)	Memilih Menu 7 setelah menghapus atau menambah data	Mengambil state teratas dari Stack dan mengembalikan data ke kondisi sebelumnya.	Sesuai Harapan	Berhasil

4.2 Kelebihan Sistem

- **Penyimpanan Data Bersifat Permanen:** Menggunakan integrasi file handling CSV sehingga data tidak hilang saat aplikasi ditutup.
- **Efisiensi Manajemen Memori:** Menggunakan alokasi memori dinamis lewat *Single Linked List*, sehingga tidak ada ruang penyimpanan yang terbuang sia-sia.
- **Keamanan Integritas Data:** Dilengkapi penanganan duplikasi nomor kamar sehingga meminimalisasi redudansi data.
- **Fleksibilitas Operasional:** Fitur *Undo* berbasis *Stack* memberikan kenyamanan bagi pengguna untuk membatalkan kesalahan aksi terakhir.

4.3 Keterbatasan Sistem

- Sistem hanya memiliki 1 sisi yaitu dari sisi pemilik kost saja → Sistem hanya dapat digunakan oleh satu pengguna yaitu hanya fitur pemilik kost saja, disini tidak ada sistem untuk penghuni kost.
- Belum terintegrasi dengan sistem pembayaran → Pengelolaan tagihan masih dilakukan secara sederhana dan belum mendukung pembayaran digital maupun pencatatan transaksi secara otomatis.
- Belum bisa menghasilkan laporan statistik → Sistem belum dapat menghasilkan laporan statistik, rekapitulasi pendapatan kost, maupun laporan penghuni secara otomatis.
- Tidak ada sistem login → Sistem ini belum memiliki sistem login yang dimana aplikasi ini hanya bisa digunakan untuk 1 pemilik kost saja.

BAB V

KENDALA DAN SOLUSI

5.1 Tabel Kendala dan Solusi

Kendala	Solusi
Program sempat bisa menyimpan data beberapa kamar dengan nomor yang sama (terjadi duplikasi data).	Menambahkan sistem validasi berbasis <i>Linked List Traversal</i> pada input data kamar, sehingga program otomatis menolak dan membatalkan proses jika nomor kamar sudah tercatat di <i>database</i> .
Mengalami kebingungan dalam memahami siklus <i>version control system</i> di GitHub, khususnya tahapan <i>pull</i> , <i>edit</i> , <i>commit</i> , dan <i>push</i> .	Meluangkan waktu untuk melakukan sesi praktik bersama secara luring. Melalui uji coba langsung, setiap anggota tim akhirnya memahami cara menarik dan mendorong pembaruan kode dengan aman.
Menulis kode secara bersamaan pada branch dan berkas utama yang sama (Code.py) sehingga sering menyebabkan merge conflict di repositori.	Kami menetapkan jadwal coding terstruktur bagi masing-masing anggota secara bergantian, serta mewajibkan setiap anggota melakukan perintah <i>pull</i> untuk mendapatkan versi kode paling mutakhir sebelum mulai melakukan pengeditan.
Porsi pengerjaan yang tidak merata pada draf awal, yang terlihat dari grafik <i>Commit History</i> di GitHub di mana kontribusi belum seimbang.	Kami segera melakukan evaluasi internal dan merombak pembagian tugas secara adil. Setiap fitur dibagi ulang porsinya agar masing-masing anggota berkontribusi langsung pada repositori, sehingga rekam jejak pengerjaan menjadi seimbang.

5.2 Pengalaman Menantang Selama Pengerjaan Proyek

Pengalaman paling menantang dalam proyek ini adalah melakukan **Refaktorisasi Sinkronisasi Paradoks Struktur Data**. Kami dituntut untuk mengimplementasikan *Single Linked List* dan *Stack* murni sebagai pusat pemrosesan data di dalam memori RAM komputer, namun di sisi lain harus tetap mempertahankan integrasi data permanen menggunakan file eksternal CSV.

Manipulasi penunjukan pointer seperti `temp = sll.head` dan `temp = temp.next` secara dinamis saat melakukan operasi CRUD dan algoritma *Bubble Sort* membutuhkan ketelitian tingkat tinggi. Kesalahan kecil pada pemutusan rantai pointer dapat menyebabkan kehilangan data (*memory leak*) atau perulangan tanpa akhir (*infinite loop*). Melalui proyek PBL ini, kami berhasil menjembatani teori struktur data abstrak dengan solusi nyata berbasis aplikasi praktis.

BAB VI

PEMBAGIAN TUGAS ANGGOTA

Nama	Tugas
Muhammad Rafly	Mengimplementasi file handling. Dimulai dari proses membaca data dari file CSV (<i>load data</i>), menyimpan data ke file CSV (<i>save data</i>), serta proses import data. Selain itu, menyusun dan mengimplementasikan struktur data <i>Single Linked List</i> sebagai penyimpanan utama data kamar, melakukan integrasi antar modul program, serta memperbaiki (<i>fix bug</i>) dan menguji program agar seluruh fitur dapat berjalan dengan baik.
Lumaris Satya Dwinanto	Membuat fitur CRUD (Create, Read, Update, Delete) pada data kamar. Selain itu, mengimplementasikan fitur Searching menggunakan algoritma Sequential Search dan Sorting menggunakan algoritma Bubble Sort sehingga pengguna dapat mencari dan mengurutkan data kamar dengan mudah.
Fahrizal Azis	Membuat tampilan tabel data menggunakan library <i>Tabulate</i> agar informasi lebih mudah dibaca. Selain itu, mengembangkan fitur Undo menggunakan struktur data Stack untuk membatalkan perubahan terakhir serta membuat fitur History yang mencatat aktivitas pengguna selama program berjalan.

BAB VII

KESIMPULAN DAN SARAN

7.1 Kesimpulan

Berdasarkan hasil perancangan dan implementasi yang telah dilakukan, keberhasilan proyek ini dapat disimpulkan dalam poin-poin berikut:

1. **Keberhasilan Implementasi Sistem Utama:** Aplikasi manajemen kost-kostan berbasis *Command Line Interface* (CLI) menggunakan bahasa Python telah sukses dirancang dan mampu menjalankan fitur CRUD (Create, Read, Update, Delete) data kamar dengan baik.
2. **Keberhasilan Penerapan Struktur Data:** Proyek ini berhasil mengintegrasikan *Single Linked List* sebagai penyimpanan data kamar secara dinamis di dalam memori, serta *Stack* yang sukses menopang berjalannya fitur pembatalan aksi (*Undo*).
3. **Keberhasilan Penyimpanan Permanen:** Penggunaan *File Handling* berformat CSV berhasil diimplementasikan, memastikan seluruh pembaruan data tersimpan secara permanen (*data persistence*) dan tidak hilang ketika program dihentikan.

7.2 Saran Pengembangan

Sistem ini memiliki peluang yang sangat baik untuk dikembangkan lebih lanjut. Beberapa fitur yang dapat ditambahkan pada masa mendatang meliputi:

1. **Fitur Autentikasi dan Multi-Pengguna (Sistem Login):** Pengembangan sistem masuk (*login*) agar aplikasi dapat digunakan oleh lebih dari satu pihak, menyediakan antarmuka dan hak akses yang berbeda antara pemilik kost dan penghuni.
2. **Fitur Pembayaran Terintegrasi:** Penambahan modul pembayaran digital atau *payment gateway* agar pengelolaan tagihan dapat direkam secara otomatis, menggantikan pencatatan transaksi yang saat ini masih manual.
3. **Fitur Pelaporan dan Rekapitulasi (Reporting):** Pembuatan fitur pembuat laporan (seperti PDF atau Excel) yang mampu menyajikan data statistik secara otomatis, seperti rekapitulasi pendapatan bulanan dan ringkasan status kamar.

DAFTAR PUSTAKA

- Dianaape. (2026). *Stack: Konsep LIFO dan penerapannya dalam dunia nyata*. Diakses pada 24 Juni 2026, dari <https://jakarta.telkomuniversity.ac.id/stack-konsep-lifo-dan-penerapannya-dalam-dunia-nyata/>
- Hernawan, M. (2023). *Searching adalah? Penjelasan arti searching untuk pemula*. Diakses pada 24 Juni 2026, dari <https://itkoding.com/searching-adalah/>
- Intern, D. (2023). *Python: Pengertian, contoh penggunaan, dan manfaat mempelajarinya*. Diakses pada 24 Juni 2026, dari <https://www.dicoding.com/blog/python-pengertian-contoh-penggunaan-dan-manfaat-mempelajarinya/>
- Kadir, A. (2014). *Pengenalan sistem informasi* (Edisi revisi). Andi.
- Rosa, A. S., & Shalahuddin, M. (2018). *Rekayasa perangkat lunak: Terstruktur dan berorientasi objek* (Edisi revisi). Informatika.
- UNESA. (n.d.). *Algoritma sort: Pengertian, jenis, cara kerja, dan contoh lengkap*. Diakses pada 24 Juni 2026, dari <https://terapan-ti.vokasi.unesa.ac.id/post/algoritma-sort-pengertian-jenis-cara-kerja-dan-contoh-lengkap>
- Widiarto, S. (2024). *CRUD: Pengertian, fungsi, contoh, dan cara membuatnya*. Diakses pada 24 Juni 2026, dari <https://www.rumahweb.com/journal/crud-adalah/>

LAMPIRAN

A. Link Github

Link: [BangRap/ASD_A_18](#)

B. Screenshot Program

```
===== MENU APLIKASI =====
1. Tambah Kamar
2. Lihat Semua Kamar
3. Cari Kamar (No/Penghuni)
4. Update Kamar (Nomor)
5. Hapus Kamar (Nomor)
6. Lihat Riwayat
7. Undo
8. Urutkan Kamar (Bubble Sort)
0. Keluar

Pilih menu: █
```

```
Pilih menu: 1
Nomor kamar: 01
Lantai: 1
Harga: 1750000
Status (Terisi/Kosong): Terisi
Penghuni: Bunatedy
Tagihan pembayaran (Rp): 990000
Kamar berhasil ditambahkan! (Tanggal mulai: 2026-06-26 )
```

Lihat semua kamar atau berdasarkan filter? (semua/filter): semua

===== DATA KAMAR =====

Nomor	Lantai	Harga	Status	Penghuni	Tanggal Mulai	Tagihan (Rp)
1	1	10	Terisi	Jokowi	2026-06-21	20
01	1	1750000	Terisi	Bunatedy	2026-06-26	990000
02	1	20000000	Kosong			
03	2	250000000	Terisi	Pria Solo	2026-06-26	1000000

Pilih menu: 2
Lihat semua kamar atau berdasarkan filter? (semua/filter): filter

Filter berdasarkan:

1. Lantai
2. Harga
3. Status

Pilih filter (1/2/3): 3
Masukkan status (Terisi/Kosong): Terisi

===== DATA KAMAR =====

Nomor	Lantai	Harga	Status	Penghuni	Tanggal Mulai	Tagihan (Rp)
1	1	10	Terisi	Jokowi	2026-06-21	20
01	1	1750000	Terisi	Bunatedy	2026-06-26	990000
03	2	250000000	Terisi	Pria Solo	2026-06-26	1000000

Pilih menu: 4
Masukkan Nomor kamar yang mau diupdate: 02

Pilih yang ingin diupdate:

1. Status & Penghuni
2. Tagihan
3. Harga

Pilihan (1/2/3): 1
Status baru (Terisi/Kosong): Terisi
Nama penghuni baru: Herlambang
Data berhasil diupdate langsung di Linked List!

Pilih menu: 5
Masukkan Nomor kamar yang dihapus: 01
Data berhasil dihapus dari Linked List!

Pilih menu: 3
Masukkan nomor kamar atau nama penghuni yang ingin dicari: Pria solo

===== Hasil Pencarian =====

Nomor	Lantai	Harga	Status	Penghuni	Tanggal Mulai	Tagihan (Rp)
03	2	250000000	Terisi	Pria Solo	2026-06-26	1000000

===== DATA KAMAR =====						
Nomor	Lantai	Harga	Status	Penghuni	Tanggal Mulai	Tagihan (Rp)
1	1	10	Terisi	Jokowi	2026-06-21	20
02	1	20000000	Terisi	Herlambang	2026-06-26	
03	2	250000000	Terisi	Pria Solo	2026-06-26	1000000
09	2	910000000	Terisi	Windah Basudara	2026-06-26	10
07	1	120000000	Terisi	Mulyadi	2026-06-26	7000000
08	2	350000	Terisi	owok	2026-06-26	120213

===== DATA KAMAR (TERURUT) =====						
Nomor	Lantai	Harga	Status	Penghuni	Tanggal Mulai	Tagihan (Rp)
1	1	10	Terisi	Jokowi	2026-06-21	20
02	1	20000000	Terisi	Herlambang	2026-06-26	
03	2	250000000	Terisi	Pria Solo	2026-06-26	1000000
07	1	120000000	Terisi	Mulyadi	2026-06-26	7000000
08	2	350000	Terisi	owok	2026-06-26	120213
09	2	910000000	Terisi	Windah Basudara	2026-06-26	10

C. Struktur Folder Proyek

Sistem Manajemen Kost-Kostan

```

|
|— Code.py
|  |— Seluruh Program
|
|— kamar.csv
|   |— Penyimpanan Data Kamar

```

D. Commit History Github

Commits on Jun 26, 2026
Fix bug, raphin menu Verified 6cd3aaa
Memasukkan link yutub Verified 70294e4
Update README.md Verified 2c22fba
Hapus file backup Verified b4946d6
Delete File penjelasan proyek Verified ba9656c
Membuat readme Verified 22b24f1
Commits on Jun 3, 2026
Update Code.py Verified b5f96b2
hapus lib Verified 3b129fd
Betulan format kamar.csv Verified 167283f
tambah tagihan pada csv dan program py, update flow fitur update Verified 9c323ba
import datetime Verified 796fa40
Commits on Jun 2, 2026
merapihkan Code.py Verified 6d58766
Update komentar Code.py Verified 02d16a9
nambahin komentar alur program 0a439b5
Commits on May 17, 2026
Penambahan fitur sorting (Bubble Sort) 6c8f9e5
Commits on May 16, 2026
validasi penginputan nomer input kamar supaya tidak duplicate data kamar 1b3d802
perbaikan integrasi sli untuk fitur update e87da0f
perbaikan integrasi SLL untuk fitur search & update 32130ba
Commits on May 16, 2026
validasi penginputan nomer input kamar supaya tidak duplicate data kamar 1b3d802
perbaikan integrasi sli untuk fitur update e87da0f
perbaikan integrasi SLL untuk fitur search & update 32130ba
Commits on May 9, 2026
fitur filter ketika read Verified f2ff7d5
ganti nama fitur Verified 6da3256
Commits on May 8, 2026
table di fungsi cari_kamar Verified c36f35b
import tabulate, read in table, fitur rename Verified 1b430d6
New Fitur (Search & History), Remove Id kamar (pakai no kamar saja), dan sedikit perbaikan bug alur code bb0ad37
Commits on May 7, 2026
tes lagi bad9ab0
tes committ c9cdece

Commits on Apr 17, 2026		
benerin	BangRap committed on Apr 17	2f6e0cb <>
Update Komentar kode	aristhekid authored on Apr 17	Verified faf9e3e <>
Update Penjelasan projek	BangRap authored on Apr 17	Verified 45051b0 <>
Update Penjelasan projek	BangRap authored on Apr 17	Verified 367f8aa <>
benerin	BangRap authored on Apr 17	Verified d582187 <>
Update Code.py	BangRap committed on Apr 17	b52c9fd <>
Merge branch 'main' of https://github.com/BangRap/PROJECT-AKHIR-ALGORITMA-DAN-STRUKTUR-DATA	BangRap committed on Apr 17	c8388aa <>
menu utama	fahri430 authored on Apr 17	Verified 0066d52 <>
Create Penjelasan projek	BangRap authored on Apr 17	Verified 24d3b29 <>
update status kamar dan delete kamar	fahri430 authored on Apr 17	Verified b9ca489 <>
Update bikin Crud bagian create & read	aristhekid authored on Apr 17	Verified 05ac5c4 <>
Commits on Apr 16, 2026		
Merge branch 'main' of https://github.com/BangRap/PROJECT-AKHIR-ALGORITMA-DAN-STRUKTUR-DATA	BangRap committed on Apr 16	6511d5e <>
Update Code.py	BangRap committed on Apr 16	0b9dbaa <>
Update Code.py	BangRap authored on Apr 16	Verified baafa41 <>
Update filehandling yang pertama error	BangRap authored on Apr 16	Verified 400a218 <>
ngebuat file csv	BangRap authored on Apr 16	Verified 835d4dd <>

E. Source Code Utama

```
# ===== LINKED LIST =====

class Node: # Class Node adalah elemen dasar dari linked list
    def __init__(self, data): # Constructor yang dijalankan saat node
        dibuat
        self.data = data # Menyimpan data kamar di dalam node
        self.next = None # Menyimpan referensi ke node berikutnya,
        walnya None karena belum tersambung ke node lain

class SingleLinkedList: # Class untuk membuat struktur data Single Linked
    List
    def __init__(self):
```

```

        self.head = None # Head adalah node pertama dalam linked list,
        awalnya None karena linked list masih kosong

    def tambah_kamar(self, kamar):
        '''
        Ini fungsi buat masukin kamar ke linkedlist
        '''

        new_node = Node(kamar) # Membuat node baru berisi data kamar
        if not self.head: # Jika linked list masih kosong
            self.head = new_node # Node baru langsung menjadi head
            return # Keluar dari fungsi karena data sudah
ditambahkan

        temp = self.head # Mulai penelusuran dari node pertama
        while temp.next: # Selama masih ada node berikutnya
            temp = temp.next # Geser temp ke node berikutnya
        temp.next = new_node # Setelah sampai node terakhir,
sambungkan node baru di bagian akhir

    def rebuild(self, data):
        '''
        Fungsi buat ngebangun ulang linked list dari list data
        '''

        self.head = None # Mengosongkan linked list terlebih dahulu
        for kamar in data: # Loop semua data kamar dari list
            self.tambah_kamar(kamar) # Masukkan satu per satu ke linked
list

# --- FITUR BARU: Konversi SLL ke List (Untuk Save ke CSV) ---
def to_list(self):
    '''
    Fungsi buat ngubah linked list ke list biasa
    '''

    hasil_list = [] # List kosong untuk menampung hasil konversi
    temp = self.head # Mulai dari node pertama
    while temp: # Selama node masih ada

```

```

        hasil_list.append(temp.data) # Ambil data dari node lalu
masukkan ke list
        temp = temp.next # Pindah ke node berikutnya
    return hasil_list # Mengembalikan hasil linked list dalam bentuk
list

# --- FITUR BARU: Hapus Node Asli SLL ---
def hapus_node_by_nomor(self, nomor_cari):
    '''
    untuk ngehapus node berdasarkan nomor kamar
    '''

    temp = self.head # Mulai dari node pertama
    prev = None # prev digunakan untuk menyimpan node sebelumnya

    if temp is not None and temp.data["nomor"] == nomor_cari: # Jika
linked list tidak kosong dan node pertama adalah data yang dicari
        self.head = temp.next # Head dipindahkan ke node berikutnya,
sehingga node pertama terhapus
        return True # Mengembalikan True karena data berhasil dihapus

    # Cari node di tengah/akhir
    while temp is not None and temp.data["nomor"] != nomor_cari: #
Selama node belum habis dan nomor kamar belum ditemukan
        prev = temp # Simpan node sekarang sebagai node sebelumnya
        temp = temp.next # Geser ke node berikutnya

    if temp is not None: # Jika data ditemukan
        prev.next = temp.next # Node sebelumnya disambungkan ke node
setelah data yang dihapus
        return True # Mengembalikan True karena data berhasil dihapus

    return False # Jika tidak ketemu, kembalikan False

# --- FITUR BARU: BUBBLE SORT PADA LINKED LIST ---
def urutkan_kamar(self):
    '''
    fungsi buat mengurutkan kamar memakai Bubble Sort pada linked list

```



```

'''

    if not self.head or not self.head.next: # Jika linked list kosong
atau cuma punya satu node
        return # Tidak perlu sorting

    diurutkan = True # Variabel penanda apakah masih ada pertukaran data
    while diurutkan: # Selama masih ada data yang ditukar
        diurutkan = False # Dianggap sudah urut dulu
        temp = self.head # Mulai pengecekan dari node pertama

        # Telusuri SLL sampai node sebelum terakhir
        while temp.next is not None: # Selama masih ada node setelah
temp

            if int(temp.data["nomor"]) > int(temp.next.data["nomor"]): #
Jika nomor kamar node sekarang lebih besar dari node berikutnya
                temp.data, temp.next.data = temp.next.data,
temp.data # Tukar isi data antar node
                diurutkan = True # Tandai bahwa masih ada pertukaran

            temp = temp.next # Geser ke node berikutnya

# ===== NEW FITUR (SEARCH & HISTORY) =====

#--- Search ---
def cari_kamar(sll): # <-- Selalu ada tambahan parameter 'sll' di sini
    '''
    fungsi buat cari kamar
    '''

    keyword = input("Masukkan nomor kamar atau nama penghuni yang ingin
dicari: ").lower()

    hasil = []
    temp = sll.head # Mulai penelusuran dari Kepala (Head) Linked List

    # Looping di linked list

```

```

while temp is not None:
    if keyword in temp.data['nomor'].lower() or keyword in
temp.data['penghuni'].lower():
        hasil.append(temp.data)
        temp = temp.next # Jalan ke node berikutnya

print("\n===== Hasil Pencarian =====")
if not hasil: #buat kalo pencariannya kosong
    print("Data yang kamu cari tidak ditemukan")
else: #atur hasil pencariannya disini
    headers = {
        "nomor": "\033[1mNomor\033[0m",
        "lantai": "\033[1mLantai\033[0m",
        "harga": "\033[1mHarga\033[0m",
        "status": "\033[1mStatus\033[0m",
        "penghuni": "\033[1mPenghuni\033[0m",
        "tanggal_mulai": "\033[1mTanggal Mulai\033[0m",
        "tagihan": "\033[1mTagihan (Rp)\033[0m"
    }
    print(tabulate.tabulate(hasil, headers=headers,
tablefmt="rounded_grid")) #disini buat ngerapihin dan membuat jadi tabel

#--- History ---
def lihat_history():
    '''
    fungsi buat ngelihat history ngedit data
    '''

    print("\n==== RIWAYAT AKTIVITAS ====")
    if not log_aktivitas: #buat Kalo aktivitasnya kosong
        print("Belum ada riwayat aktivitas")
    else: #Disini atur hasil aktivitas
        for i, log in enumerate(log_aktivitas, 1):
            print(f"{i}.{log}")

# ===== CRUD =====

```

```

def sort_kamar(sll):
    '''
    fungsi buat mengurutkan data kamar kost
    '''

    global data

    if not sll.head: #Buat ngecek linkedlistnya kosong apa engga
        print("\n Data kosong, tidak ada yang bisa diurutkan!")
        return

    print("\nSedang mengurutkan kamar berdasarkan nomor...")

    sll.urutkan_kamar() # buat jalanin algoritma Bubble Sort di dalam SLL

    data = sll.to_list() # Buat sinkronin ke list global abis diurutin

    save_data(data) #buat nyimpen data ke csv
    log_aktivitas.append("Mengurutkan daftar kamar berdasarkan nomor")

    print(" Kamar berhasil diurutkan dari nomor terkecil ke terbesar!")

    # Hasil sorting
    headers = {
        "nomor": "\033[1mNomor\033[0m",
        "lantai": "\033[1mLantai\033[0m",
        "harga": "\033[1mHarga\033[0m",
        "status": "\033[1mStatus\033[0m",
        "penghuni": "\033[1mPenghuni\033[0m",
        "tanggal_mulai": "\033[1mTanggal Mulai\033[0m",
        "tagihan": "\033[1mTagihan (Rp)\033[0m"
    }

    print("\n===== DATA KAMAR (TERURUT)
=====")

    print(tabulate.tabulate(data, headers=headers, tablefmt="rounded_grid"))

def tambah_kamar(sll):

```

```

'''
fungsi buat nambah kamar
'''

global data
nomor = input("Nomor kamar: ")

# ===== Validasi Nomer =====
temp = sll.head # dicek dulu dari head linked list
while temp is not None:
    if temp.data["nomor"] == nomor:
        print(f"\n Error: Kamar nomor {nomor} sudah ada! Silahkan input
nomor kamar yang lain.")
        return # disini langsung keluar dari fungsi, tidak jadi
menambahkan data kamar karna sudah terinput
    temp = temp.next # geser ke node berikutnya
# =====

lantai = input("Lantai: ")
harga = input("Harga: ")
status = input("Status (Terisi/Kosong): ").capitalize()

if status.lower() == "kosong":
    # Jika kamar kosong, skip penghuni, tanggal_mulai, dan tagihan
    penghuni = ""
    tanggal_mulai = ""
    tagihan = ""
    print("Status Kosong: penghuni, tanggal mulai, dan tagihan
dikosongkan otomatis.")
else:
    penghuni = input("Penghuni: ")

# Validasi input tagihan: harus berupa angka
while True:
    tagihan_input = input("Tagihan pembayaran (Rp): ")
    if tagihan_input.isdigit():
        tagihan = tagihan_input
        break
    else:
        print(" Error: Tagihan harus berupa angka! Coba lagi.")

```

```

        # Tanggal mulai diisi otomatis dari datetime saat data diinput,
        bukan dari user

        tanggal_mulai = datetime.datetime.now().strftime("%Y-%m-%d ")

    kamar = {
        "nomor": nomor,
        "lantai": lantai,
        "harga": harga,
        "status": status,
        "penghuni": penghuni,
        "tanggal_mulai": tanggal_mulai,
        "tagihan": tagihan
    }

    history_stack.append([d.copy() for d in data]) #Simpen state data
    sebelum ditambah (buat undo)

    data.append(kamar)
    log_aktivitas.append(f"Menambahkan kamar nomor {nomor}")
    save_data(data)
    sll.tambah_kamar(kamar)

    if tanggal_mulai:
        print(f"Kamar berhasil ditambahkan! (Tanggal mulai:
{tanggal_mulai})")
    else:
        print("Kamar berhasil ditambahkan!")

def lihat_kamar():
    '''
    fungsi buat lihat data kamar kost
    '''

    global data

    if not data: #Ngecek data kosong apa engga

```

```

        print("Data kosong")
        return

    headers = {
        "nomor": "\033[1mNomor\033[0m",
        "lantai": "\033[1mLantai\033[0m",
        "harga": "\033[1mHarga\033[0m",
        "status": "\033[1mStatus\033[0m",
        "penghuni": "\033[1mPenghuni\033[0m",
        "tanggal_mulai": "\033[1mTanggal Mulai\033[0m",
        "tagihan": "\033[1mTagihan (Rp)\033[0m"
    }

    pilihan = input("Lihat semua kamar atau berdasarkan filter?
(semua/filter): ").lower()

    # ini if else buat milih lihat semua kamar atau ngefilter
    if pilihan == "semua":
        tampil = data

    elif pilihan == "filter":
        print("""
Filter berdasarkan:
1. Lantai
2. Harga
3. Status
""")
        jenis_filter = input("Pilih filter (1/2/3): ")

        tampil = []

        # Disini buat if else filter kamar
        if jenis_filter == "1":
            lantai = input("Masukkan lantai (1/2): ")
            tampil = [k for k in data if k["lantai"] == lantai]

        elif jenis_filter == "2":
            harga = input("Masukkan harga yang dicari: ")

```

```

        tampil = [k for k in data if k["harga"] == harga]

    elif jenis_filter == "3":
        status = input("Masukkan status (Terisi/Kosong): ").capitalize()
        tampil = [k for k in data if k["status"].lower() ==
status.lower()]

    else:
        print("Pilihan filter tidak valid!")
        return

    if not tampil:
        print("Tidak ada kamar yang dicari.")
        return

    else:
        print("Pilihan tidak valid!")
        return

    print("\n===== DATA KAMAR =====")
    print(tabulate.tabulate(tampil, headers=headers,
tablefmt="rounded_grid"))

def update_kamar(sll):
    '''
    fungsi buat ngeupdate data kamar kost
    '''
    global data

    nomor_cari = input("Masukkan Nomor kamar yang mau diupdate: ")

    temp = sll.head #ini dari head SLL dulu

    while temp is not None:
        if temp.data["nomor"] == nomor_cari:
            # Simpan state lama ke stack sebelum data diubah (untuk Undo)
            history_stack.append([d.copy() for d in data])

```

```

        print("""
Pilih yang ingin diupdate:
1. Status & Penghuni
2. Tagihan
3. Harga
""")

        pilih_update = input("Pilihan (1/2/3): ")

        if pilih_update == "1":
            # Status ditanya duluan, penghuni menyesuaikan
            status_baru = input("Status baru (Terisi/Kosong): ")
            status_baru = status_baru.capitalize()

            if status_baru.lower() == "kosong":
                log_aktivitas.append(f"Update kamar {nomor_cari}: status
-> Kosong, penghuni '{temp.data['penghuni']}' dikosongkan")
                temp.data["status"] = "Kosong"
                temp.data["penghuni"] = ""
                temp.data["tanggal_mulai"] = ""
            else:
                penghuni_baru = input("Nama penghuni baru: ")
                tanggal_mulai_baru =
datetime.datetime.now().strftime("%Y-%m-%d ")
                log_aktivitas.append(f"Update kamar {nomor_cari}: status
-> {status_baru}, penghuni {temp.data['penghuni']} -> {penghuni_baru}")
                temp.data["status"] = status_baru
                temp.data["penghuni"] = penghuni_baru
                temp.data["tanggal_mulai"] = tanggal_mulai_baru

        elif pilih_update == "2":
            while True:
                tagihan_baru = input("Tagihan pembayaran baru (Rp): ")
                if tagihan_baru.isdigit():
                    break
                else:
                    print(" Error: Tagihan harus berupa angka! Coba
lagi.")

                log_aktivitas.append(f"Update kamar {nomor_cari}: tagihan
{temp.data.get('tagihan', '-')} -> {tagihan_baru}")

```



```

        temp.data["tagihan"] = tagihan_baru

    elif pilih_update == "3":
        while True:
            harga_baru = input("Harga baru (Rp): ")
            if harga_baru.isdigit():
                break
            else:
                print(" Error: Harga harus berupa angka! Coba
lagi.")

            log_aktivitas.append(f"Update kamar {nomor_cari}: harga
{temp.data.get('harga', '-')} -> {harga_baru}")
            temp.data["harga"] = harga_baru

        else:
            print("Pilihan tidak valid! Update dibatalkan.")
            history_stack.pop() # Batalkan undo yang sudah disimpan
            return

    # 3. Sinkronkan ke list global dan simpan ke CSV
    data = sll.to_list()
    save_data(data)

    print("Data berhasil diupdate langsung di Linked List!")
    return

    temp = temp.next # Pindah ke kamar berikutnya

    print("Nomor kamar tidak ditemukan!")

def hapus_kamar(sll):
    '''
    fungsi buat ngehapus data kamar kost
    '''

    global data
    nomor_cari = input("Masukkan Nomor kamar yang dihapus: ")

```

```

# 1. Simpan riwayat untuk Undo (WAJIB sebelum diubah)
history_stack.append([d.copy() for d in data])

# 2. Lakukan penghapusan murni menggunakan Linked List
berhasil_hapus = sll.hapus_node_by_nomor(nomor_cari)

if berhasil_hapus:
    log_aktivitas.append(f"Menghapus kamar nomor {nomor_cari}")

    # 3. Sinkronkan global data dengan kondisi SLL yang baru
    data = sll.to_list()

    # 4. Save ke CSV
    save_data(data)
    print("Data berhasil dihapus dari Linked List!")
else:
    # Jika tidak ketemu, batalkan undo yang tadi disimpan
    history_stack.pop()
    print("Nomor kamar tidak ditemukan!")

# ===== STACK UNDO =====

def undo(sll):
    """
    fungsi buat ngeundo perubahan data
    """

    global data

    if history_stack:
        #Perbaikan: pop data terakhir dari stack
        data = history_stack.pop()
        log_aktivitas.append("Melakukan Undo")
        save_data(data)
        sll.rebuild(data)

```

```
        print("Undo berhasil!")

    else:
        print("Tidak ada riwayat untuk undo.")
```